手写小GPT学习笔记

Day11承接Day10的Multi-Head Attention，搭建GPT解码器的**单层Transformer Block**，核心新增残差连接、LayerNorm层、Feed Forward Network（前馈网络）三大组件，解决深度模型训练退化、梯度消失问题，实现注意力特征的深度加工，是搭建完整GPT模型的关键过渡。

一、Transformer Block 核心作用

Transformer Block是GPT解码器的基础重复单元，GPT-2/GPT-3就是堆叠N个相同的Block实现深度语义建模。它将**多头注意力**、**残差连接**、**层归一化**、**前馈网络**整合为一个模块，实现：

- **特征提取**：通过多头注意力捕捉序列全局/局部语义关联
- **稳定训练**：残差连接缓解深度网络梯度消失，LayerNorm加速收敛、防止梯度爆炸
- **特征升华**：前馈网络对注意力输出做非线性变换，提升模型表达能力
- **可堆叠性**：输入输出维度完全一致，可无限堆叠加深模型，适配不同规模任务

> 💡 **GPT解码器专属结构**：仅含**掩码多头自注意力**（无Encoder-Decoder注意力），遵循**Pre-LN**架构（归一化放在子层之前，训练更稳定）。

> 💡 **Mac MPS设备适配说明**：PyTorch MPS后端暂不支持带digits参数的round操作，修复方案：要么启用CPU回退，要么打印前将张量迁移至CPU，两种方案均不影响模型训练推理。

二、核心组件原理详解

1. 残差连接（Residual Connection）

残差连接是解决深度网络退化的核心设计，公式为：$\text{Output} = x + \text{SubLayer}(x)$

逻辑：将子层（注意力/FFN）的输入直接加到输出上，即便子层学习到零映射，模型也能保留原始特征，避免梯度消失，让深度堆叠成为可能。在Transformer Block中，**注意力子层、FFN子层各配一个残差分支**。

2. 层归一化（Layer Normalization）

区别于Batch Norm，LayerNorm针对**单个样本的特征维度**做归一化，适配NLP变长序列场景，公式：

$$\text{LayerNorm}(x) = \gamma \cdot \frac{x - \mu}{\sigma + \epsilon} + \beta$$

- μ：单个样本特征均值，σ：单个样本特征标准差
- γ、β：可学习的缩放、偏移参数，保留特征表达能力
- ϵ：极小值（1e-5），防止分母为0

GPT采用**Pre-LN**架构：归一化放在子层之前，而非之后，训练更稳定、收敛更快。

3. 前馈网络（Feed Forward Network, FFN）

FFN是逐位置独立的全连接网络，对每个位置的注意力特征做非线性变换，公式：

$$\mathrm{FFN}(x) = \mathrm{Dropout}(\mathrm{Linear}_2(\mathrm{GELU}(\mathrm{Linear}_1(x))))$$

- **Linear1升维逻辑**：将d_model映射为4×d_model（GPT标准配置），升维多出的3倍维度并非固定值，而是**可学习的权重参数**，初始值由PyTorch默认初始化规则生成（线性层默认采用**Xavier均匀初始化**，兼顾梯度稳定，防止训练初期参数过大/过小）；升维后维度扩容，用于捕捉更丰富的非线性特征，全程参数随训练迭代更新。
- **GELU激活函数**：平滑曲线型激活函数，形似**带弧度的缓坡滑梯**，输入趋近负无穷时缓慢趋近0，输入增大时平滑爬升趋近原值，全程无尖锐拐点、梯度连续不中断；对比ReLU（像直角滑梯，负数区间直接砸到地面归零、梯度骤降为0），GELU无梯度清零问题，训练更稳定，适配Transformer深层网络。
- Linear2：降维层，将4×d_model还原为d_model，保证维度匹配残差连接
- Dropout：防止过拟合，随机丢弃部分神经元

4. 完整Block计算流程（Pre-LN架构）

1. 输入特征先做LayerNorm，再进入多头自注意力层，输出+输入做残差连接
2. 残差结果再做LayerNorm，进入FFN层，输出+残差结果做二次残差连接
3. 最终输出作为下一个Block的输入，维度始终保持d_model不变

三、完整代码实现（复用Day9、Day10组件）

1. 依赖导入与基础组件（复用+补全）

基础依赖+复用Day9/10核心代码

```python
import torch
import torch.nn as nn
import math
import torch.nn.functional as F
import os

# ------------------- MPS设备兼容配置（Mac专用） -------------------
# 方案1：启用MPS运算失败自动回退CPU（推荐，无需改代码逻辑）
os.environ["PYTORCH_ENABLE_MPS_FALLBACK"] = "1"
```

```python

# ------------------- 复用Day9：掩码函数 -------------------
def create_causal_mask(seq_len, device=None):
    mask = torch.triu(torch.ones(seq_len, seq_len, device=device), diagonal=1).bool()
    return mask.unsqueeze(0)

def create_attn_padding_mask(seq, pad_idx):
    batch_size, seq_len = seq.shape
    mask = (seq == pad_idx).unsqueeze(1)
    return mask.expand(batch_size, seq_len, seq_len)

# ------------------- 复用Day10：缩放点积注意力 -------------------
class ScaledDotProductAttention(nn.Module):
    def __init__(self, dropout=0.1):
        super().__init__()
        self.dropout = nn.Dropout(dropout)

    def forward(self, q, k, v, mask=None):
        d_k = q.size(-1)
        scores = torch.matmul(q, k.transpose(-2, -1)) / math.sqrt(d_k)

        if mask is not None:
            scores = scores.masked_fill(mask, -1e9)

        # 重点：softmax在前，dropout在后，保证权重和为1
        attn_weights = torch.softmax(scores, dim=-1)
        attn_weights = self.dropout(attn_weights)
        output = torch.matmul(attn_weights, v)

        return output, attn_weights

# ------------------- 复用Day10：多头注意力 -------------------
class MultiHeadAttention(nn.Module):
    def __init__(self, d_model, num_heads, dropout=0.1):
        super().__init__()
        assert d_model % num_heads == 0, "d_model必须能被num_heads整除"

        self.d_model = d_model
        self.num_heads = num_heads
        self.d_k = d_model // num_heads

        self.w_q = nn.Linear(d_model, d_model)
        self.w_k = nn.Linear(d_model, d_model)
        self.w_v = nn.Linear(d_model, d_model)
        self.w_o = nn.Linear(d_model, d_model)

```

```python
        self.attention = ScaledDotProductAttention(dropout=dropout)
        self.dropout = nn.Dropout(dropout)

    def split_heads(self, x, batch_size):
        x = x.view(batch_size, -1, self.num_heads, self.d_k)
        return x.transpose(1, 2)

    def combine_heads(self, x, batch_size):
        x = x.transpose(1, 2)
        return x.contiguous().view(batch_size, -1, self.d_model)

    def forward(self, q, k, v, mask=None):
        batch_size = q.size(0)

        q_proj = self.w_q(q)
        k_proj = self.w_k(k)
        v_proj = self.w_v(v)

        q_split = self.split_heads(q_proj, batch_size)
        k_split = self.split_heads(k_proj, batch_size)
        v_split = self.split_heads(v_proj, batch_size)

        if mask is not None:
            mask = mask.unsqueeze(1)

        attn_output, attn_weights = self.attention(q_split, k_split, v_split, mask=mask)
        output = self.combine_heads(attn_output, batch_size)
        output = self.dropout(self.w_o(output))

        return output, attn_weights
```

2. 新增：FFN层 + Transformer Block实现

Day11新增核心代码

```python
# ------------------- Day11新增：前馈网络FFN -------------------
class FeedForwardNetwork(nn.Module):
    def __init__(self, d_model, dropout=0.1, expansion_factor=4):
        super().__init__()
        # GPT标准配置：升维4倍
        self.linear1 = nn.Linear(d_model, d_model * expansion_factor)
        self.linear2 = nn.Linear(d_model * expansion_factor, d_model)
        self.dropout = nn.Dropout(dropout)
        # GELU激活函数，适配Transformer训练
        self.gelu = nn.GELU()
```

```python

    def forward(self, x):
        # 升维 -> 激活 -> Dropout -> 降维
        x = self.linear1(x)
        x = self.gelu(x)
        x = self.dropout(x)
        x = self.linear2(x)
        return x

# ------------------- Day11核心：Transformer Block -------------------
class TransformerBlock(nn.Module):
    def __init__(self, d_model, num_heads, dropout=0.1, expansion_factor=4):
        super().__init__()
        # 1. 多头注意力层
        self.attn = MultiHeadAttention(d_model, num_heads, dropout)
        # 2. 前馈网络
        self.ffn = FeedForwardNetwork(d_model, dropout, expansion_factor)
        # 3. 两个层归一化（Pre-LN架构）
        self.norm1 = nn.LayerNorm(d_model)
        self.norm2 = nn.LayerNorm(d_model)
        # 4. Dropout层
        self.dropout = nn.Dropout(dropout)

    def forward(self, x, mask=None):
        """
        前向传播
        Args:
            x: 输入特征，shape [batch_size, seq_len, d_model]
            mask: 因果掩码+填充掩码，shape [batch_size, seq_len, seq_len]
        Returns:
            x: Block输出特征，shape不变
            attn_weights: 注意力权重【仅用于调试/可视化/分析，不参与前向计算】
        """
        # 子层1：多头注意力 + 残差 + Pre-LN
        residual = x
        x = self.norm1(x)  # Pre-LN：先归一化
        attn_out, attn_weights = self.attn(x, x, x, mask)  # 自注意力
        x = residual + self.dropout(attn_out)  # 残差连接

        # 子层2：FFN + 残差 + Pre-LN
        residual = x
        x = self.norm2(x)  # Pre-LN：先归一化
        ffn_out = self.ffn(x)
        x = residual + self.dropout(ffn_out)  # 残差连接

        return x, attn_weights
```

3. 测试代码（MPS设备适配版，修复round报错）

Transformer Block测试（Mac MPS兼容）

```python
if __name__ == "__main__":
    # 设备配置：自动适配CUDA/MPS/CPU
    if torch.cuda.is_available():
        device = torch.device("cuda")
    elif torch.backends.mps.is_available():
        device = torch.device("mps")
    else:
        device = torch.device("cpu")
    print(f"使用设备：{device}")

    # 超参数（对齐Day10）
    batch_size = 2
    seq_len = 5
    d_model = 512
    num_heads = 8
    pad_idx = 0
    dropout = 0.1

    # 模拟输入：序列嵌入特征
    x = torch.randn(batch_size, seq_len, d_model).to(device)
    # 模拟真实序列（含PAD）
    seq = torch.tensor([[1,2,3,0,0], [4,5,0,0,0]]).to(device)

    # 构建掩码
    causal_mask = create_causal_mask(seq_len, device)
    padding_mask = create_attn_padding_mask(seq, pad_idx)
    mask = torch.logical_or(causal_mask, padding_mask)

    # 初始化Block
    block = TransformerBlock(d_model, num_heads, dropout).to(device)

    # 前向传播
    output, attn_weights = block(x, mask)

    # 验证维度一致性（核心：输入输出维度完全相同）
    print(f"\n输入形状：{x.shape}")
    print(f"输出形状：{output.shape}")
    print(f"注意力权重形状：{attn_weights.shape}")
    print(f"\n✅ Transformer Block验证通过：维度匹配，可堆叠训练！")

    # 验证注意力权重合理性（每行和为1）
    print("\n第一个样本第一个头注意力权重求和（每行≈1）:")
    # 方案2：打印前将张量迁移至CPU，彻底规避MPS不支持round的问题（通用兜底）
```

```
    attn_sum = attn_weights[0, 0].sum(dim=-1).to("cpu").round(2)
    print(attn_sum)
```

四、关键细节与避坑指南

1. Pre-LN vs Post-LN 差异

- **Pre-LN（本笔记采用）**：归一化在子层前，训练更稳定，适合深层GPT模型，梯度更易传播
- **Post-LN（原始Transformer）**：归一化在子层后，深层模型易梯度消失，不推荐用于GPT

2. 残差连接顺序

必须遵循：**残差 = 输入 → 子层处理 → Dropout → 加残差**，Dropout作用在子层输出上，避免破坏原始特征。

3. 注意力权重和不为1的原因

测试时权重和略偏差属于正常现象：**Dropout随机丢弃权重**，导致softmax归一化被破坏；推理阶段关闭Dropout后，权重和严格为1。

4. 注意力权重attn_weights的作用

核心定位：纯调试/分析工具，不参与模型前向计算、不参与梯度回传，相当于模型的“黑盒透视镜”，主要用途有：

- **调试纠错**：查看掩码是否生效（比如因果掩码是否屏蔽未来位置、填充掩码是否屏蔽PAD位），判断注意力分布是否符合预期，排查代码bug。
- **模型可解释性**：可视化注意力热图，看模型“关注哪里”（比如生成文本时，模型是否聚焦前文关键词），理解模型决策逻辑。
- **训练监控**：观察权重分布是否异常（比如全聚焦某一位置、权重均匀无差别），辅助调参（头数、学习率等）。
- **科研/演示**：用于论文绘图、教学演示，直观展示Transformer的注意力机制原理。

实际部署/训练时，可直接删掉返回值，不影响模型功能；保留它是为了学习阶段更易理解、排查问题。

5. 维度匹配红线

- FFN的输入输出必须等于d_model，否则无法做残差连接
- LayerNorm的维度必须和d_model一致，针对最后一维（特征维度）归一化

6. Mac MPS设备避坑要点

- MPS不支持 `round(digits=2)` ，但**模型核心运算（注意力、矩阵乘法）完全支持**，无需强制切CPU
- 优先用**环境变量回退**：无需改代码，PyTorch自动把不兼容算子丢CPU跑，速度几乎无感知
- 兜底方案：打印/可视化前，用 `.to("cpu")` 把张量迁移到CPU，再做round、绘图等操作
- 训练阶段可删掉round打印语句，完全不影响模型训练，仅调试时需要

五、关键注意事项

1. **激活函数选择**：GPT优先用GELU，禁用ReLU，避免负区间梯度清零
2. **Dropout位置**：仅作用在子层输出，不作用在残差分支和归一化层
3. **掩码传递**：Block的掩码需传递给多头注意力，保证因果性、屏蔽PAD位置
4. **堆叠逻辑**：多个Block串联时，前一个Block的输出直接作为后一个的输入，无需修改维度
5. **初始化技巧**：线性层采用Xavier初始化，防止训练初期梯度爆炸

六、与前后学习内容衔接

1. **前置依赖**：Day9掩码机制、Day10多头注意力，是Block的核心子模块

2. **后续进阶**：Day12将堆叠N个Transformer Block，搭配Embedding层、Linear分类头，实现**完整GPT模型**，并编写训练脚本完成文本生成任务

🎯 至此，GPT解码器的核心单元已实现，掌握Block的堆叠逻辑，即可快速搭建任意规模的小GPT模型！

（注：文档部分内容可能由 AI 生成）